

DATA STRUCTURES & ANALYSIS OF ALGORITHMS

Exam-Ready Notes & Answers

Prepared from the supplied Unit 1 / Unit 2 study material and matched to the September 2026 mid-semester question paper.

Focus: Algorithm Analysis • Arrays • Sparse Matrices • Stacks • Queues • Circular Queues • Deques • Infix/Postfix/Prefix

PART 1 — ALGORITHM ANALYSIS

1. What is an Algorithm? Give its characteristics.

Definition: An algorithm is a finite, step-by-step sequence of clear and unambiguous instructions used to solve a particular problem and produce the desired output.

Characteristics:

- Input
- Output
- Definiteness / Unambiguity
- Finiteness
- Effectiveness / Feasibility
- Independence from a particular programming language

Example — addition of two numbers:

1. START
2. Read A and B
3. $C \leftarrow A + B$
4. Display C
5. STOP

2. Why is Algorithm Design Efficiency Important?

Algorithm efficiency determines how much **time** and **memory** an algorithm requires as the input size increases.

- Reduces execution time.
- Reduces memory consumption.
- Allows the program to handle large inputs.
- Helps compare different algorithms.
- Improves scalability and overall performance.

Example: Linear Search is $O(n)$, whereas Binary Search is $O(\log n)$ on sorted data.

3. Linear Search vs Binary Search

Linear Search	Binary Search
Checks elements sequentially	Divides the search range into halves
Does not require sorted data	Requires sorted data
Simple to implement	More complex
Worst case: $O(n)$	Worst case: $O(\log n)$
Best case: $O(1)$	Best case: $O(1)$
Suitable for small/unordered data	Suitable for large sorted data

4. Resource / Space Complexity

Space complexity is the total memory required by an algorithm during execution.

Fixed space: code, constants, simple variables and fixed-size data structures.

Variable space: dynamic allocation and recursion call-stack space that can depend on input size.

Formula: $S(P) = C + S(I)$

5. Time Complexity

Time complexity represents the amount of computational time required by an algorithm to complete execution. It is generally represented as **$T(n)$** , where n is the input size.

Example: if an operation takes n steps, $T(n) = c \times n$, so the growth is linear: **$O(n)$** .

6. Best Case, Average Case and Worst Case

Case	Meaning
Best Case	Minimum time required for execution
Average Case	Average/expected time over possible inputs
Worst Case	Maximum time required for execution

7. Linear Search — Best, Worst and Average Case

Case	Situation	Complexity
Best	Target is at the first position	$O(1)$
Average	Target is randomly distributed	$O(n)$
Worst	Target is last or absent	$O(n)$

For a successful random search, the average number of comparisons is approximately $(n + 1) / 2$.

8. Performance of Linear and Non-Linear Data Structures

Linear: Elements are arranged sequentially. Examples: arrays, stacks, queues and linked lists.

Non-linear: Elements are arranged hierarchically or as interconnected networks. Examples: trees and graphs.

Linear	Non-Linear
Sequential arrangement	Hierarchical/network arrangement
Generally single sequential traversal	Multiple possible traversal paths
Array, Stack, Queue, Linked List	Tree, Graph

9. Primitive vs Non-Primitive Data Structures

Primitive	Non-Primitive
Basic data types	Complex structures derived from primitive types
int, float, char, boolean	Array, Linked List, Stack, Queue, Tree, Graph
Usually stores a single value	Stores/organizes collections of values

10. Static vs Dynamic Data Structures

Static	Dynamic
Size is fixed	Size can grow/shrink at runtime
Example: Array	Example: Linked List
Fast/simple access	Flexible memory utilization
May waste memory or overflow	Has pointer/memory-management overhead

11. Important Properties of an Algorithm

- Input
- Output
- Definiteness
- Finiteness
- Effectiveness

12. Common Issues in Writing an Algorithm

- Ambiguous instructions
- Infinite loops / missing termination condition
- Improper boundary or edge-case handling
- Excessive time consumption
- Excessive memory usage
- Correctness versus performance trade-offs

Avoid them by: defining steps clearly, specifying termination conditions, testing boundary cases, dry-running examples, and analyzing time/space complexity.

13. Asymptotic Notations

Notation	Meaning
Big-O, O	Upper bound; commonly used for worst-case growth
Big-Omega, Ω	Lower bound; commonly associated with best-case growth
Big-Theta, Θ	Tight bound / exact order of growth
Little-o, o	Strict upper bound
Little-omega, ω	Strict lower bound

14. A Priori and A Posteriori Analysis

A Priori Analysis	A Posteriori Analysis
Performed before execution	Performed after implementation/execution
Theoretical	Empirical
Independent of specific hardware	Depends on machine, processor, compiler, etc.
Uses time/space complexity	Uses actual running time/resource measurements

PART 2 — ARRAYS & SPARSE MATRICES

15. Single-Dimensional Array

A one-dimensional array is a collection of elements of the same data type stored in sequential memory locations.

```
int A[5] = {10, 20, 30, 40, 50};
```

Advantages: direct/random access, simple implementation, efficient sequential storage and easy traversal.

Disadvantage: a normal static array has a fixed size.

16. Two-Dimensional Array

A two-dimensional array stores elements in rows and columns.

```
int A[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Row-major order: 1 2 3 4 5 6

Column-major order: 1 4 2 5 3 6

17. Storage Mechanism of 1-D and 2-D Arrays

1-D array:

$LOC(A[i]) = BA + i \times w$

where BA = base address and w = size of each element.

2-D array, row-major:

$LOC(A[i][j]) = BA + [(i \times \text{number_of_columns}) + j] \times w$

18. Sparse Matrix with Example

A sparse matrix is a matrix in which the majority of elements are zero. Instead of storing every zero, only non-zero elements can be stored.

Original 4×4 matrix:

```
0 0 0 5  
0 8 0 0  
0 0 0 0  
2 0 0 0
```

3-Tuple representation:

Row	Column	Value
4	4	3
0	3	5
1	1	8
3	0	2

The first row stores matrix metadata: total rows, total columns and number of non-zero elements.

PART 3 — STACK

19. Define Stack

A stack is a non-primitive linear data structure in which insertion and deletion take place at the same end called the **TOP**. It follows **LIFO (Last In, First Out)**.

```
TOP
↓
| 30 |
| 20 |
| 10 |
```

20. Main Stack Operations

- **PUSH** — inserts an element at the top.
- **POP** — removes an element from the top.

Other operations include checking whether the stack is empty/full and peeking at the top element.

21. Algorithm for PUSH

```
Algorithm PUSH(stack[], MAXSIZE, top)
```

```
If top == MAXSIZE - 1
    Print "Stack is full"
    Return
```

```
Read item
top = top + 1
stack[top] = item
```

22. Algorithm for POP

```
Algorithm POP(stack[], top)
```

```
If top == -1
    Print "Stack is empty"
    Return
```

```
item = stack[top]
top = top - 1
```

```
Return item
```

23. Stack vs Queue

Stack	Queue
LIFO	FIFO
Insertion at top	Insertion at rear
Deletion at top	Deletion at front
Push / Pop	Enqueue / Dequeue

PART 4 — INFIX, POSTFIX & PREFIX

24. Infix, Postfix and Prefix

Notation	Operator Position	Example
Infix	Between operands	A + B
Postfix	After operands	AB+
Prefix	Before operands	+AB

Example: $A + B * C$

Postfix = **ABC*+**

Prefix = **+A*BC**

25. Infix → Postfix Conversion Algorithm

1. Create an empty operator stack.
2. Scan the expression from left to right.
3. If the symbol is an operand, add it to postfix.
4. If it is '(', push it.
5. If it is ')', pop until '(' is found, then discard '('.
6. If it is an operator:
 Pop higher/equal-precedence operators from the stack
 and append them to postfix.
 Push the current operator.
7. At the end, pop all remaining operators.

Precedence: ^ (highest), then *, /, %, then + and -.

26. Example — Infix to Postfix

Convert: **A + B * C**

A → A
+ → stack
B → AB
* → stack
C → ABC
End → pop * and +

Answer: **ABC*+**

Convert: **(A+B)*C**

Answer: **AB+C***

27. Evaluate a Postfix Expression

Create an empty operand stack.

Scan from LEFT to RIGHT.

If operand:
 PUSH it.

If operator:
 op2 = POP()
 op1 = POP()
 result = op1 operator op2
 PUSH(result)

After scanning, POP the final result.

Example: $23+ \rightarrow 2 + 3 = 5$

28. Evaluate a Prefix Expression

Create an empty stack.

Scan from RIGHT to LEFT.

If operand:

PUSH it.

If operator:

op1 = POP()

op2 = POP()

result = op1 operator op2

PUSH(result)

After scanning, the stack contains the final result.

Example: $+ / 8 2 3 = (8/2) + 3 = 7$

PART 5 — QUEUES

29. Define Queue

A queue is a non-primitive linear data structure in which insertion takes place at the **REAR** and deletion takes place at the **FRONT**. It follows **FIFO (First In, First Out)**.

```
FRONT → 10  20  30  40 ← REAR
```

30. Main Queue Operations

- Enqueue / Qinsert — inserts an element at the rear.
- Dequeue / Qdelete — deletes an element from the front.

Other operations include Create Queue, Queue Full and Queue Empty.

31. Array Implementation of Queue

```
int Q[MAXSIZE];  
int Front = -1;  
int Rear = -1;
```

Two indices are maintained: **Front** and **Rear**. Initially Front = Rear = -1.

32. Enqueue Algorithm

```
Algorithm Qinsert(Q[], MAXSIZE, Rear, Front, item)
```

```
If Rear >= MAXSIZE - 1  
    Print "Queue is full"  
    Return
```

```
Rear = Rear + 1  
Q[Rear] = item
```

```
If Front == -1  
    Front = 0
```

33. Dequeue Algorithm

```
Algorithm Qdelete(Q[], Front, Rear)
```

```
If Front == -1  
    Print "Queue is empty"  
    Return
```

```
Item = Q[Front]
```

```
If Front == Rear  
    Front = Rear = -1  
Else  
    Front = Front + 1
```

```
Return Item
```

34. Linear Queue vs Circular Queue

Linear Queue	Circular Queue
Rear moves only forward	Rear wraps around
May waste freed spaces	Reuses freed spaces
Can suffer false overflow	Avoids this problem
Simple implementation	Uses modulo arithmetic
rear = rear + 1	rear = (rear + 1) % MAXSIZE

35. Circular Queue

A circular queue connects the last array position back to the first position. It uses modulo arithmetic to reuse freed locations.

```
Full condition:  
(Rear + 1) % MAXSIZE == Front
```

```
Empty condition:  
Front == -1
```

```
Advance Rear:  
Rear = (Rear + 1) % MAXSIZE
```

```
Advance Front:  
Front = (Front + 1) % MAXSIZE
```

36. Advantages of Circular Queue

- Better memory utilization.
- Reuses positions freed after deletion.
- Avoids false overflow.
- No shifting of elements is required.
- Efficient insertion and deletion.
- Useful for buffers and continuous data processing.

37. Circular Queue Insertion Algorithm

```
Algorithm CQinsert(Q[], Front, Rear, item, MAXSIZE)
```

```
If Front == (Rear + 1) % MAXSIZE  
    Print "Queue is full"  
    Return
```

```
If Front == -1  
    Front = 0
```

```
Rear = (Rear + 1) % MAXSIZE  
Q[Rear] = item
```

38. Circular Queue Deletion Algorithm

```
Algorithm CQdelete(Q[], Front, Rear, MAXSIZE)
```

```
If Front == -1  
    Print "Queue is empty"  
    Return
```

```
Item = Q[Front]
```

```
If Front == Rear  
    Front = Rear = -1
```

```
Else  
    Front = (Front + 1) % MAXSIZE
```

```
Return Item
```


PART 7 — SEPTEMBER 2026 QUESTION PAPER ANSWERS

PART A — 2-Mark Answers

44. Q1 — Define an algorithm. List any two essential properties.

An algorithm is a finite sequence of unambiguous, step-by-step instructions used to solve a computational problem.

Any two: **Definiteness** — every instruction must be clear and unambiguous. **Finiteness** — the algorithm must terminate after a finite number of steps.

45. Q2 — What is a structure in C++? Give a simple example.

A structure (struct) is a user-defined data type that groups related variables, possibly of different data types, under one name.

```
struct Student {  
    int rollNo;  
    char name[20];  
    float marks;  
};
```

```
Student s1;
```

46. Q3 — Give two examples each for static and dynamic data structures.

Static: Array, static stack.

Dynamic: Linked List, dynamic stack/queue using linked lists.

47. Q4 — Define a stack. What are its two main operations?

A stack is a linear data structure following LIFO (Last In, First Out). Its two main operations are **Push** and **Pop**.

48. Q5 — Define a queue. Mention its two main operations.

A queue is a linear data structure following FIFO (First In, First Out). Its two main operations are **Enqueue** and **Dequeue**.

49. Q6 — Differentiate between stack and queue.

Stack	Queue
LIFO	FIFO
Insert/delete at top	Insert at rear, delete at front
Push and Pop	Enqueue and Dequeue

PART B — 5-Mark Answers

50. Q7 — Common issues while writing an algorithm and how to avoid them

- Ambiguous steps — use clear and precise instructions.
- Infinite loops — provide correct termination conditions.
- Boundary errors — test first/last/empty cases.
- Poor efficiency — analyze time and space complexity.
- Incorrect logic — dry-run with sample inputs.
- Excessive resource usage — choose suitable data structures and algorithms.

51. Q8 — Best, worst and average case of Linear Search

Case	Example	Complexity
Best	Target at first element	O(1)
Average	Target randomly positioned	O(n)
Worst	Target at last element or absent	O(n)

52. Q9 — Structures vs Classes in C++

Structure	Class
Declared using struct	Declared using class
Members are public by default	Members are private by default
Can contain data and functions	Can contain data and functions
Often convenient for grouped data	Commonly used for encapsulation

The structure/class comparison above is standard C++ terminology; the supplied notes did not contain a dedicated section on this topic.

53. Q10 — Algorithm to evaluate a postfix expression

```
Create an empty operand stack.
Scan from left to right.
If operand → PUSH.
If operator:
    op2 = POP()
    op1 = POP()
    result = op1 operator op2
    PUSH(result)
At the end, POP the final result.
```

54. Q11 — Differentiate between linear and circular queue

A linear queue moves its rear only toward the end of the array, while a circular queue wraps around to the beginning using modulo arithmetic. Circular queues reuse freed spaces and avoid false overflow.

55. Q12 — Stack operations Push and Pop with algorithms

Push: Check overflow, increment top, insert item.

```
If top == MAXSIZE - 1 → Overflow
Else:
    top = top + 1
    stack[top] = item
```

Pop: Check underflow, retrieve top item, decrement top.

```
If top == -1 → Underflow
Else:
    item = stack[top]
    top = top - 1
    return item
```

PART C — 10-Mark Answers

56. Q13 — Short notes: 1-D arrays, 2-D arrays and sparse matrices

1-D Array: Same-type elements stored sequentially. Example: `int A[5] = {10,20,30,40,50}`.

2-D Array: Elements arranged in rows and columns. Example: `int A[2][3] = {{1,2,3},{4,5,6}}`.

Sparse Matrix: Matrix containing mostly zeros. A 3-tuple representation stores Row, Column and Value for non-zero entries, with an optional metadata row.

57. Q14 — Infix to Postfix conversion with example

Algorithm:

1. Create an empty operator stack.
2. Scan left to right.
3. Operand → append to output.
4. '(' → push.
5. ')' → pop until '('; discard '('.
6. Operator → pop higher/equal precedence operators, then push current operator.
7. Pop all remaining operators at the end.

Example:

A + B * C → ABC*+

58. Q15 — Array implementation of queue with Enqueue and Dequeue

Declaration:

```
int Q[MAXSIZE];  
Front = -1;  
Rear = -1;
```

ENQUEUE:

If Rear == MAXSIZE - 1 → Queue Full

Else:

```
    Rear = Rear + 1  
    Q[Rear] = item  
    If Front == -1:  
        Front = 0
```

DEQUEUE:

If Front == -1 → Queue Empty

Else:

```
    item = Q[Front]  
    If Front == Rear:  
        Front = Rear = -1  
    Else:  
        Front = Front + 1  
    Return item
```

FINAL — QUICK REVISION SHEET

Topic	Remember
Algorithm	Finite, clear step-by-step procedure
Time Complexity	Time/resources as a function of input size
Space Complexity	Memory required during execution
Stack	LIFO — Push, Pop
Queue	FIFO — Enqueue, Dequeue
Linear Search	Best $O(1)$, Average $O(n)$, Worst $O(n)$
Binary Search	Requires sorted data; $O(\log n)$ worst case
Infix	$A+B$
Postfix	$AB+$
Prefix	$+AB$
Circular Queue Full	$(Rear+1)\%N == Front$
Circular Queue Empty	$Front == -1$
Sparse Matrix	Mostly zero; store non-zero values
Static DS	Fixed size
Dynamic DS	Resizable at runtime
Linear DS	Array, Stack, Queue, Linked List
Non-linear DS	Tree, Graph

Highest-priority exam topics: Infix → Postfix, Stack Push/Pop, Queue Enqueue/Dequeue, Linear vs Circular Queue, Linear Search cases, 1-D/2-D arrays, Sparse Matrix, Circular Queue algorithms, Prefix/Postfix evaluation.